

C++ Template 延伸自主練習作業

主題：STL 與基礎資料結構入門

1. 自主學習主題與目標

自主學習的核心主題為「C++泛型程式設計（Templates）的延伸應用與標準模板庫（STL）基礎資料結構入門」。本自主學習練習作業接續 C++ 中 Function Template 與 Class Template 的核心觀念，進一步延伸至標準模板庫中的 STL 與標準演算法的整合實作，從第一週的 STL 基礎操作練習，到第二週以「學生成績管理系統」為題的應用小專題，進而深入理解 Templates 及 STL 的核心機制，為大二的專業課程「資料結構」先備知識基礎。

本專案的目標著重於理解 Template 與 STL 的內在關係，從「Template 可讓函式與類別套用於不同型別」的底層觀念出發，進一步認識 C++ 標準模板函式庫的運作機制，並理解 `std::vector` 與 `std::string` 等常用工具在本質上皆為模板的具體應用。透過第一週的自主探索與第二週的小專題程式撰寫，熟練掌握 `vector`、`stack`、`queue` 與 `map` 等常見 STL 容器的成員函式調用與記憶體基本操作，培養出「資料結構」的架構思維，提前熟悉動態資料結構的運作邏輯，能更好的對接大二的專業課程，為資料結構與演算法建立先備優勢。

2. Template 與 STL 的關係

STL 的本質就是一個高度抽象化的「泛型元件庫」。若沒有 Template 的支持，我們在撰寫陣列或鏈結串列時，就必須針對 `int`、`double`、`string` 甚至是自訂的 `struct` 各自寫一套完全重複的程式碼。Template 允許開發者將「資料型別」抽離成一個參數化型別，使得同一套資料結構或演算法可以服務於任何型別。

當我們宣告 `vector<int>` 或 `vector<Student>` 時，編譯器會在編譯期根據我們傳入的型別參數，動態生成對應型別的具體程式碼。這種機制的優點在於毫無執行期（Runtime）的效能損失，完美貫徹了 C++ 「不為用不到的功能付出代價」的設計哲學。不論是容器（如 `vector`）還是演算法（如 `sort`），它們的底層架構全部都是泛型模板的實踐。

3.常見 STL 容器用途整理。

在第一週的自主學習中，我摸索了 C++ 標準模板庫 (STL) 中的數種核心容器。以下是常見的四大 STL 容器之底層資料結構、時間複雜度特性以及核心適用場景。

3.1 `std::vector` (動態陣列)

3.1.1 底層資料結構

動態連續陣列。在記憶體中佔用一段連續的空間。

3.1.2 時間複雜度

支援 $O(1)$ 的隨機存取。在尾端進行插入與刪除的平均時間複雜度為 $O(1)$ ；但在陣列中間或開頭進行插入與刪除時，因為需要搬移後續的所有元素，時間複雜度會退化至 $O(N)$ 。

3.1.3 特性與最使用場景

`std::vector` 是 C++ 中最通用且預設首選的順序型容器。它具備記憶體局部性的優勢，快取命中率高。最適合用於資料量大小動態不固定，但操作上以頻繁讀取、隨機存取為主，且極少在陣列中間進行隨機插入或刪除的場景。

3.2 `std::stack` (堆疊)

3.2.1 底層資料結構

屬於容器配接器，預設底層通常封裝了 `std::deque` (雙向佇列) 或 `std::vector` 來進行記憶體管理。

3.2.2 時間複雜度

不支援任意位置的隨機存取。對於頂端 (Top) 元素的插入 (Push) 與彈出 (Pop) 操作，時間複雜度嚴格保證為 $O(1)$ 。

3.2.3 特性與最使用場景

具備後進先出的典範特性。新加入的資料會壓在最頂端，最先被讀取的也是最晚加入的資料。這種結構非常適合用於具備回溯性質的演算法，包括編譯器中的括號配對檢查、網頁瀏覽器的「上一頁」功能，以及將遞迴函式轉化為非遞迴實作的深度優先搜尋。

3.3 `std::queue`（佇列）

3.3.1 底層資料結構

同樣屬於容器配接器，預設底層通常由 `std::deque` 或 `std::list`。

3.3.2 時間複雜度

不支援隨機存取。在隊尾插入元素（Push）以及在隊首彈出元素（Pop）的時間複雜度皆為高效的 $O(1)$ 。

3.3.3 特性與最使用場景

一樣具備先進先出的公平特性。最先進入容器的元素會最先被處理，完美模擬了現實生活中的排隊行為。包括作業系統的核心排程調度、伺服器接收網路封包的緩衝佇列以及圖形演算法中的廣度優先搜尋節點走訪。

3.4 `std::map`（關聯式字典）

3.4.1 底層資料結構

自平衡二元搜尋樹，具體實作為紅黑樹（Red-Black Tree）。

3.4.2 時間複雜度

無論是資料的插入、刪除還是尋找（Lookup），其時間複雜度皆穩定保持在對數時間的 $O(\log N)$ 。

3.4.3 特性與最使用場景

專門用來儲存具有內在關聯的 Key-Value 資料，且容器內部的 Key 絕對不重複，並會自動依照 Key 的大小進行字典序排列。它非常適合用於需要建立對映關係、依關鍵字高效檢索，或需要維持資料有序性的複合場景。包括英文句子中的單字出現次數統計、學號與學生資料的對應查表，以及各式字典檔的建置。

4.第 2 週小專題設計說明

本專案實作的「學生成績管理系統」在整合結構體與 STL 容器，建構出一個具備高防呆性與統計功能的命令列管理工具，學生資料可選擇使用 `vector<Student>` 或 `map<string, Student>` 儲存。經過評估後，本專案最終決定選用 `std::vector<Student>`。

4.1 選用 `std::vector<Student>` 理由

雖然 `std::map` 在透過學號 (Key) 查詢時擁有 $O(\log N)$ 的高速查找優勢，但本系統的核心功能包含了「依照成績進行排序」。若使用 `map`，其底層會強制依學號字典序排序，要改成依成績排序必須將整棵樹的節點拷貝至 `vector` 後才能呼叫 `sort()`。為了減少型別轉換的開銷，直接採用 `vector` 配合 Lambda 匿名函式進行排序，在邏輯表達與效能權衡上更為直覺。

4.2 功能模組架構設計

4.2.1 防呆學號重複檢查

新增學生時，程式會先行啟動唯讀走訪，比對現有學號。若發現重複，則攔截並拒絕新增，維護資料庫的正確性。

4.2.2 多模態雙向排序

打破傳統單向排序的限制，允許使用者互動式選擇由高到低（遞減）或由低到高（遞增）排序。

4.2.3 泛型挑戰題整合

自主設計 `getMax` 與 `getMin` 函式模板，並完美嵌入統計模組，在走訪全班分數的同時，動態篩選出最高分與最低分。

5.主要程式架構與重要程式片段

5.1 泛型函式模板(Function Template)的定義與原理

為了實現跨型別的數值比較，設計了 getMax 與 getMin 模板函式。這兩個函式不僅能比較 int，還能直接用於 double 或 float 的比較。

```
15  template <class T>
16      T getMax(T a, T b)
17      {
18          return(a>b) ? a : b;
19      }
20
21  template <class T>
22      T getMin(T a, T b)
23      {
24          return(a<b) ? a : b;
25      }
```

5.2 For 迴圈與自動型別推導(學號重複檢查)

在功能 1 新增學生資料中，使用了 Range-based for loop 與 auto。

```
52      if(choice==1)
53      {
54          string addID;
55          cout<<"輸入學號 > ";
56          cin>>addID;
57
58          bool duplicate=false;
59
60          for(const auto& s : students)
61          {
62              if(s.id==addID)
63              {
64                  duplicate=true;
65                  break;
66              }
67          }
68          if(duplicate)
69          {
70              cout<<"該學生資料已存在!返回主選單!"<<endl<<endl;
71          }
```

若寫成 for(auto s : students)，每一輪走訪都會引發一次 Student 結構的複製行為，當資料量龐大時會嚴重拖慢速度。加上引用符號 const auto&，程式會直接透過記憶體位址讀取原資料，同時受到 const 保護不被誤改。

5.3 Lambda 匿名函式的雙向自訂排序

在功能 3 中，利用標準庫<algorithm>的 std::sort 演算法，搭配自訂的 Lambda 表達式作為排序準則。

```
102     else if(choice==3)
103     {
104         if(students.empty())
105         {
106             cout<<"目前無學生資料!!!返回主選單!"<<endl<<endl;
107         }
108         else
109         {
110             int type;
111             cout<<"請選擇排序方式!(1 成績由高到低 or 2 成績由低到高) > ";
112             cin>>type;
113
114             if(type==1)
115             {
116                 sort(students.begin(), students.end(), [](const Student& a, const Student& b)
117                 {
118                     return (a.score>b.score);
119                 });
120                 cout<<"排序完成!可使用功能2查看全部學生成績!"<<endl<<endl;
121             }
122             else if(type==2)
123             {
124                 sort(students.begin(), students.end(), [](const Student& a, const Student& b)
125                 {
126                     return (a.score<b.score);
127                 });
128                 cout<<"排序完成!可使用功能2查看全部學生成績!"<<endl<<endl;
129             }
130             else
131             {
132                 cout<<"輸入排序方式錯誤!返回返回主選單!";
133             }
134         }
135     }
```

5.4 統計成績模組與 Template 函式的具體呼叫

在功能 5 統計模組中，將全班學生的分數逐一累加，並在走訪過程中即時呼叫我們自訂的 getMax 與 getMin 泛型模板。

```
168     else if(choice==5)
169     {
170         if(students.empty())
171         {
172             cout<<"目前無學生資料!!!返回主選單!"<<endl<<endl;
173         }
174         else
175         {
176             int total=0;
177             int pass=0;
178             int fail=0;
179             int max=students[0].score;
180             int min=students[0].score;
181
182             for(const auto& s : students)
183             {
184                 total=total+s.score;
185                 if(s.score>=60)
186                 {
187                     pass=pass+1;
188                 }
189                 else
190                 {
191                     fail=fail+1;
192                 }
193
194                 max=getMax(max,s.score);
195                 min=getMin(min,s.score);
196             }
197             cout<<"=====統計成績===== "<<endl;
198             cout<<"全班平均: "<<(double)total/students.size()<<" 分"<<endl;
199             cout<<"最高分: "<<max<<" 分"<<endl;
200             cout<<"最低分: "<<min<<" 分"<<endl;
201             cout<<"及格人數: "<<pass<<" 人"<<endl;
202             cout<<"不及格人數: "<<fail<<" 人"<<endl;
203             cout<<endl;
204         }
205     }
```

6. 執行畫面或輸出結果

```
歡迎使用學生成績管理系統！

=====學生成績管理系統功能清單=====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

請選擇功能 > 
```

初始介面

```
請選擇功能 > 2
目前無學生資料!!!返回主選單！

=====學生成績管理系統功能清單=====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

請選擇功能 > 
```

如無資料無法使用功能，則返回主選單

```
請選擇功能 > 1
輸入學號 > S11459011
輸入學生姓名 > 莊詠恩
輸入學生成績 > 100

學生資料新增成功!!!
```

功能 1，新增學生資料

```
請選擇功能 > 1
輸入學號 > S11459011
該學生資料已存在!返回主選單！

=====學生成績管理系統功能清單=====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

請選擇功能 > 
```

如重複輸入相同學號，將無法新增

```
請選擇功能 > 2
學生資料:
學號: S11459011 | 姓名:莊詠恩 | 成績: 100
學號: S11459000 | 姓名:測試一 | 成績: 60
學號: S11459001 | 姓名:測試二 | 成績: 59
```

功能 2，查看所有學生資料

```
請選擇功能 > 3
請選擇排序方式!(1 成績由高到低 or 2 成績由低到高) > 2
排序完成!可使用功能2查看全部學生成績!
```

功能 3，將學生資料以遞減或遞增排序

```
請選擇功能 > 2
學生資料:
學號: S11459001 | 姓名:測試二 | 成績: 59
學號: S11459000 | 姓名:測試一 | 成績: 60
學號: S11459011 | 姓名:莊詠恩 | 成績: 100
```

排序成功

```
請選擇功能 > 3
請選擇排序方式!(1 成績由高到低 or 2 成績由低到高) > 0
輸入排序方式錯誤!返回主選單!

=====學生成績管理系統功能清單=====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能 > |
```

如輸入錯誤排序方式，則返回主選單

```
請選擇功能 > 4
輸入查詢學號 > S11459011
找到學生資料!
姓名: 莊詠恩 | 成績: 100
```

功能 4，使用學號查詢學生資料

```
請選擇功能 > 4
輸入查詢學號 > S11459002
查無此學生資料!!!返回主選單!

=====學生成績管理系統功能清單=====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能 > |
```

如查無此學號，則返回主選單

```
請選擇功能 > 5
=====統計成績=====
全班平均: 73 分
最高分: 100 分
最低分: 59 分
及格人數: 2 人
不及格人數: 1 人
```

功能 5，統計全班資料

```
=====學生成績管理系統功能清單=====
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

請選擇功能 > 6
輸入錯誤!!!請重新輸入!
```

如輸入錯誤的功能，將要求重新輸入

7. GitHub Repo Link 與檔案結構說明

GitHub Repo Link : <https://github.com/mark1018-gg/Student-Grade-Management-System>

檔案結構

—— main.cpp	// 核心原始程式碼檔案 (包含結構、Template、主迴圈選單)
—— README.md	// 專案技術說明文件 (詳述專題功能、編譯與執行環境)
—— report.pdf	// 本篇學習結案報告

8.遇到的問題與解決方式

初次接觸 struct 與 template 語法的思維轉變

在過去的程式設計經驗中，我主要處理的都是獨立且單一的基礎資料型別（如個別的 int、string 變數）。這次的期末自主學習專題，必須去理解並應用以前比較少接觸的兩個核心語法，結構（struct）與泛型函式模板（template）。過程中，我花費了比較多的時間去理解它們的底層邏輯、語法規範以及在既有選單架構中的正確使用方式，期間也遇到了不少具體的思維痛點。

8.1 泛型模板（template）語法的抽象性

8.1.1 問題

在「Template Function 延伸挑戰題」中。雖然作業提供了 `template <class T>` 的宣告範例，但「泛型」這個概念對初學者來說非常抽象。在我的既定認知中，任何函式的參數都必須在編譯前清清楚楚地指定型別。當看到代稱 `T` 時，我一開始完全不知道該如何在主程式中正確地「呼叫」它，甚至在統計最高分與最低分時，卡在不知道如何把結構內的成員變數 `s.score` 正確的傳入 `getMax` 與 `getMin` 函式中。

8.1.2 解決方式

為了解決這個問題，我上網查了 C++ 關於「編譯期具現化（Instantiation）」的運行邏輯。原來當我在主程式的功能 5 中寫下 `getMax(max, s.score)` 時，因為 `max` 和 `s.score` 都是整數（int），編譯器就會自動在幕後幫我複製一份函式，把所有的 `T` 替換成 `int`。理解了這個底層運作原理後，語法就不再顯得高深莫測。也順利篩選出正確的最高與最低分數。

9.學習心得與未來可延伸方向

9.1 學習心得

這兩週的自主學習與整合專題實作，顛覆了我過去對於 C++ 程式設計的既定認知。在過去接觸 C++ 時，我習慣於處理獨立的變數與靜態陣列，這使得程式碼在面對複雜的複合資料時顯得極為臃腫且難以維護。這次專題強迫我走出舒適圈，深入去理解與應用以前比較少接觸的自訂結構與泛型模板語法。體會到 struct 解決了資料「橫向封裝」的痛點，讓學號、姓名與成績能以高度關聯的物件型態在容器中流動，完美解決了排序時資料不同步的災難；而 template 則解決了演算法「縱向跨型別」的複用問題，讓我明白如何寫出與資料型別無關的通用最大與最小值函式。這次的實作磨練為我銜接大二的核心學門「資料結構與演算法」建立了先備觀念與信心。

9.2 未來可延伸方向

雖然目前的學生成績管理系統已經滿足了作業的基本規範與防呆檢查，但在實務應用與軟體工程的標準下，仍有許多極具價值的延伸重構空間。首先在資料持久化方面，目前系統運行的所有資料都儲存在隨機存取記憶體中，一旦程式關閉，輸入的學生資料便會全數消失，未來可規劃引入 C++ 的 `<fstream>` 檔案輸入輸出串流標頭檔，在使用者選擇離開系統前自動將資料庫匯出為標準的 CSV 或 TXT 檔案，並在下一次程式啟動時自動載入，使系統具備真正的資料實用與延續價值。